

SyllabusOS

Product Motivation & Context

Students operate in a fragmented information environment where syllabi, assignment PDFs, and announcements are dense, inconsistent, and easy to misinterpret. The result is missed deadlines, poor prioritization, and reactive rather than planned workflows. Existing tools, such as calendars and task managers, require manual input and assume the student already understands course expectations. Emerging AI tools can parse a syllabus once, but they do not stay synced when professors change deadlines mid-semester via email or class announcements.

SyllabusOS converts unstructured academic documents into structured, actionable plans, and keeps those plans synced with professor announcements across the term. The opportunity is high because every student faces this problem every semester, the input data already exists, and no existing tool closes the loop between the original syllabus and in-semester changes. Success in the short term is accurate extraction, usable schedules, and reliable change detection. Long-term success is improved academic outcomes, reduced stress, and daily habit integration. Key tradeoffs include extraction accuracy versus speed, and simplicity versus feature depth. Overbuilding early risks complexity without adoption.

Target Users & Problems

User Segments

- Undergraduate students managing multiple concurrent courses
- First-year and international students unfamiliar with syllabus conventions
- Students in high-workload programs like engineering or pre-medical tracks
- Students balancing academics with external commitments like work or athletics

Core Pain Points

1. Deadlines and requirements are embedded in dense, unstructured documents
2. Grading weights are not easily interpretable, leading to misaligned priorities
3. Assignment scope and required effort are underestimated
4. Workload fluctuates unpredictably across the term
5. Academic support resources like office hours are underutilized

User Journey Considerations:

1. Design a landing page that uses a sample student persona to demonstrate how SyllabusOS works end-to-end
 - Introduce a fictional USC student persona (name, year, major, course load)
 - Show their problem: 5 syllabi, different formats, deadlines scattered everywhere
 - Walk through how SyllabusOS solves it: upload → extract → calendar → announcement sync
 - Include sample screenshots or UI mockups of the actual product
2. Interactive calendar dashboard as the core home screen
 - Heatmap view showing workload intensity across the semester (light = easy weeks, red = crunch weeks)
 - Click any week to expand and see individual assignments, exams, and projects

- Click any specific task to dig deeper — see weight, confidence level, raw source text from syllabus
 - Toggle between single course view and aggregated multi-course view
 - "Heaviest week" and "this week" highlights surface automatically
- 3. After landing page, user can add a profile and customize their SyllabusOS experience
 - Collect basic info: name, university, major, year
 - Set default term dates so they don't re-enter each upload
 - Set notification preferences for upcoming deadlines
 - Save course color preferences for the calendar view
- 4. Onboarding flow after profile setup
 - Prompt user to upload their first syllabus right after profile creation
 - Progress indicator: Profile done → Syllabus uploaded → Calendar populated
 - Surface a "review flagged items" prompt if low-confidence extractions exist
- 5. Interaction
 - Users can make small edits and track task completion.
- 6. Ongoing Use
 - Daily check-in flow: User opens app → sees today's tasks plus a 3-day horizon → marks tasks complete or defers them.
 - Weekly planning flow: A Sunday prompt surfaces next week's workload heatmap → user adjusts study blocks or flags overload weeks.
 - Change detection flow: System monitors forwarded course announcement emails → flags any deadline or exam date that differs from the parsed syllabus → user accepts or overrides → downstream tasks reflow accordingly.
 - Mid-semester course change flow: User adds a dropped or added course → system re-aggregates across all courses and flags new conflicts.
 - Exam-period mode: In the final two weeks of the term, the system surfaces a prep schedule weighted by remaining grade impact per course.

Solution & Product Design

MVP (Week 3): The MVP focuses on immediate utility through accurate parsing and clear presentation. It reduces manual effort and enables rapid adoption without requiring behavioral change.

1. Document upload or text input interface
2. NLP-based extraction of deadlines, assignments, grading weights, exam schedules, late-work policies, and office hours
3. Structured representation of extracted information confidence flags on uncertain extractions
4. Automated calendar generation with a weekly view
5. Basic workload estimation using rule-based classification of assignment types
6. Workload forecasting with identification of peak periods
7. Adaptive study planning based on deadlines and grading weights
8. Announcement change detection (MVP differentiator): Users forward course announcement emails to a dedicated address; the system diffs announced dates against the parsed syllabus and flags discrepancies.
9. Lightweight notification system: Daily email digest of upcoming tasks and any detected changes.

Final Product (Week 4-5)

1. Push notifications and mobile-optimized reminder system
2. Aggregation across multiple courses with conflict detection
3. Task decomposition for complex assignments (rule-based splitting on numbered deliverables)
4. Direct Brightspace/LMS integration (beyond manual email forwarding)
5. Feedback mechanisms to refine workload estimation models
6. Integration with external calendar platforms (Google Calendar export) — v2
7. Integration with gradebooks to update grades and priorities in real time — v2
8. Potential feature: Create a paid tier that lets students layer in personal events like work shifts or practice schedules — so the calendar reflects their full life, not just academics. Paid users get intelligent conflict alerts, like flagging that a project deadline falls on a game day or that finals week overlaps with peak work hours, helping student-athletes and part-time workers plan ahead before it's too late.

Design Considerations

Design decisions are grounded in the four user segments defined above:

- For first-year and international students (unfamiliar with syllabus conventions):
 - Structured View includes hover-tooltips on academic terms (midterm, office hours, late policy, drop deadline).
 - Onboarding does not assume prior knowledge of US academic scheduling conventions.
 - Extracted policies are shown in plain language ("You can submit homework up to 2 days late with a 10% penalty") rather than raw syllabus text.
- For high-workload engineering and pre-medical students:
 - Calendar surfaces conflict density via a weekly workload heatmap so crunch weeks are visible 2+ weeks in advance, not the night before.
 - Priority sorting defaults to grade-weighted impact rather than chronological order.
 - Exam weeks trigger a prep-mode view that blocks study time proportional to each exam's weight.
- For students balancing academics with work or athletics:
 - Users can enter recurring non-academic commitments (shifts, practice, games, travel).
 - The adaptive study planner routes study blocks around these commitments rather than scheduling on top of them.
 - Conflict detection includes non-academic blocks, not just assignment overlap.
- For undergraduates managing multiple concurrent courses:
 - Each course gets distinct color coding but appears in a unified deadline stream, so users do not context-switch between per-course pages.
 - Cross-course conflict detection flags weeks where multiple heavy deliverables land together.
 - Multi-course aggregation is the default view; per-course deep-dive is one click away.

Metrics & Success

Metric	Definition
Activation Rate	% of users who upload a syllabus and generate a schedule
Weekly Active Users	Users actively using the product each week
Week 4 Retention	% of users returning after one month
Parsing Accuracy	% of correctly extracted deadlines and tasks
On-Time Task Completion Rate	% of planned tasks completed on time
Change Detection Precision	% of flagged syllabus changes reflecting real deadline shift

GTM, Risks & Summary

GTM

Initial distribution focuses on student networks for fast adoption.

- Target: College students (starting with immediate class/community)
- Channels: Class groups, student orgs, word-of-mouth
- Core Value: Instant syllabus → structured schedule that stays synced with professor announcements/emails, with no manual effort

Risks (+ potential solutions)

- Parsing accuracy on grading weights (MVP feature #2): Regex extraction will fail on syllabi that express weights in tables or prose ("Homework comprises one-third of the final grade"). *Mitigation:* Fall back to TF-IDF + Naive Bayes classifier on sentences containing percentage or fraction tokens. Surface low-confidence extractions with a visual flag in Structured View so users correct them before calendar generation.
- Date resolution on relative references (MVP feature #4): Dates like "Week 5" or "the Monday after Spring Break" cannot resolve without a course start date. *Mitigation:* Require term start/end input during onboarding. Flag unresolvable relative dates as "needs review" rather than silently guessing.
- Announcement change detection false positives (MVP feature #8): Email forwarding may pick up announcements that mention dates but do not change deadlines ("Remember, the midterm is next week"). *Mitigation:* Only flag when the detected date differs from an existing parsed deadline for the same assignment. Require user confirmation before updating the calendar. Log false positives to tune detection rules.
- Overengineering on task decomposition (Final Product feature #3): Decomposing complex assignments into subtasks has unclear scope and could consume a full development week. *Mitigation:* Hard cutoff; if rule-based decomposition (splitting on numbered deliverables or "Part 1 / Part 2" patterns) is not producing useful output by end of Week 4, cut the feature entirely rather than extending into ML-based approaches.
- User trust on first-run parsing: Incorrect extractions on a user's first upload will kill retention. *Mitigation:* Confidence flags on every extracted field, inline editing on the Structured View page, and a "re-parse" button. Never silently commit a low-confidence extraction to the calendar.
- Time constraints on dev scope: 2–3 weeks to MVP, 2 more to final is tight. *Mitigation:* Favor simple, reliable methods (regex + rule-based before ML). Any Final Product feature not prototyped by mid-Week 4 is cut to v2.

Summary

The overall goal is to simplify academic planning by converting unstructured syllabi into actionable schedules that stay synced throughout the term. Our MVP will prioritize speed, usability, and announcement-change detection to drive quick adoption and differentiation, while maintaining a feasible development scope.

Technical Design

Our technical design prioritizes simplicity, speed of implementation, and alignment with concepts covered in class (text preprocessing, TF-IDF, basic classification, and structured data handling).

1. Input & Preprocessing Layer

- Accept input via:
 - PDF upload (converted to text using a basic parser)
 - Direct text input
- Apply preprocessing steps:
 - Lowercasing
 - Removing non-alphabetic characters
 - Tokenization
 - Stopword removal

2. Information Extraction (Core NLP Logic)

Given time constraints, use a hybrid approach:

Rule-Based Extraction (Primary)

- Regex patterns for:
 - Dates (e.g., MM/DD, "October 5")
 - Keywords: "Exam", "Homework", "Project", "Due"
- Simple heuristics:
 - Sentences containing both a keyword + date → likely an assignment

Potential ML Enhancement (if time allows)

- Use TF-IDF Vectorizer + simple classifier (Naive Bayes) to classify sentences into:
 - Assignment
 - Exam
 - Policy

3. Data Structuring Layer

- Store extracted information in a Pandas DataFrame:
 - Columns: task_name, type, due_date, weight, notes
- This should enable:
 - Easy sorting/filtering
 - Simple aggregation across courses
 - Clean integration with visualization

4. Calendar & Workload Generation

- Convert extracted deadlines into:
 - Weekly grouped tasks
 - Basic calendar-style table (likely won't be full UI integration)
- Workload estimation (rule-based):
 - Exams → high workload

- Projects → medium/high
- Homework → low

5. Change Detection Layer (new)

- Dedicated email inbox receives forwarded course announcements.
- Extraction pipeline runs the same date/keyword regex + classifier on announcement text.
- Detected dates are diffed against the stored DataFrame for that course:
 - Match with different date → flag as "potential change" for user review
 - New assignment not in syllabus → flag as "new item"
 - No match / ambiguous → log but do not surface

6. Frontend / Output

- Interactive webpage
 - Home page displays schedule/to-do list in either calendar or list form
 - Home page also has area to upload new syllabus
 - Each syllabus/class has an individual page with more granular breakdown of priorities, grades, OH, policies
 - Final product: ability to go inside an assignment to see breakdown of tasks
 - Ability to manually edit priorities or add other commitments
 - Separate page for personalized study plan
 - Notification panel surfacing flagged changes from the change-detection layer

7. System Flow Summary

1. User uploads syllabus
2. Text is preprocessed
3. Rules + simple ML extract key info
4. Data stored in a structured format (DataFrame)
5. Calendar/tasks generated and displayed
6. User forwards course announcements; change-detection layer diffs against stored deadlines
7. User confirms or overrides any flagged changes; calendar reflows

8. Design Tradeoffs

- Accuracy vs Speed:
 - Rule-based methods are less flexible but faster to implement
- Complex Models vs Feasibility:
 - Avoid CNN/LSTM models due to time and data requirements
- Automation vs Control:
 - Provide basic editing to compensate for imperfect extraction
- Change detection precision vs recall:
 - Favor precision, false positives erode trust faster than missed changes. Require user confirmation for all flagged updates.